

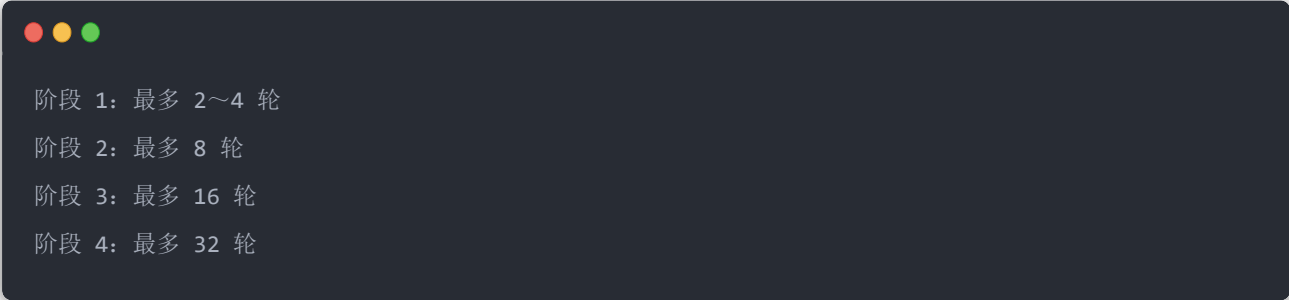
“多轮 RL + 课程学习” 已经是一条相当明确的研究路线。只是不同论文中的“课程”不一定指题目从易到难，还可能是逐渐增加交互轮数、工具数量、环境复杂度，或者动态选择模型当前成功率适中的任务。

1. 最直接的代表：AgentGym-RL / ScalingInter-RL

[AgentGym-RL: Training LLM Agents for Long-Horizon Decision Making through Multi-Turn Reinforcement Learning](#) 提出的 ScalingInter-RL，是目前最贴近“多轮长度课程学习”的工作。它的核心做法是逐步扩大允许的最大交互轮数：

$$H_1 < H_2 < \dots < H_K,$$

例如：



```
阶段 1: 最多 2~4 轮  
阶段 2: 最多 8 轮  
阶段 3: 最多 16 轮  
阶段 4: 最多 32 轮
```

训练早期只允许短轨迹，使模型先学会局部操作和短期信用分配；随着策略能力增强，再开放更长的轨迹，让模型学习规划、纠错和恢复。为什么直接训练长轨迹容易失败？假设每一轮选择正确动作的概率为 p ，完成 H 轮任务的概率约为：

$$P(\text{success}) \approx p^H.$$

即便单轮正确率达到 0.8：

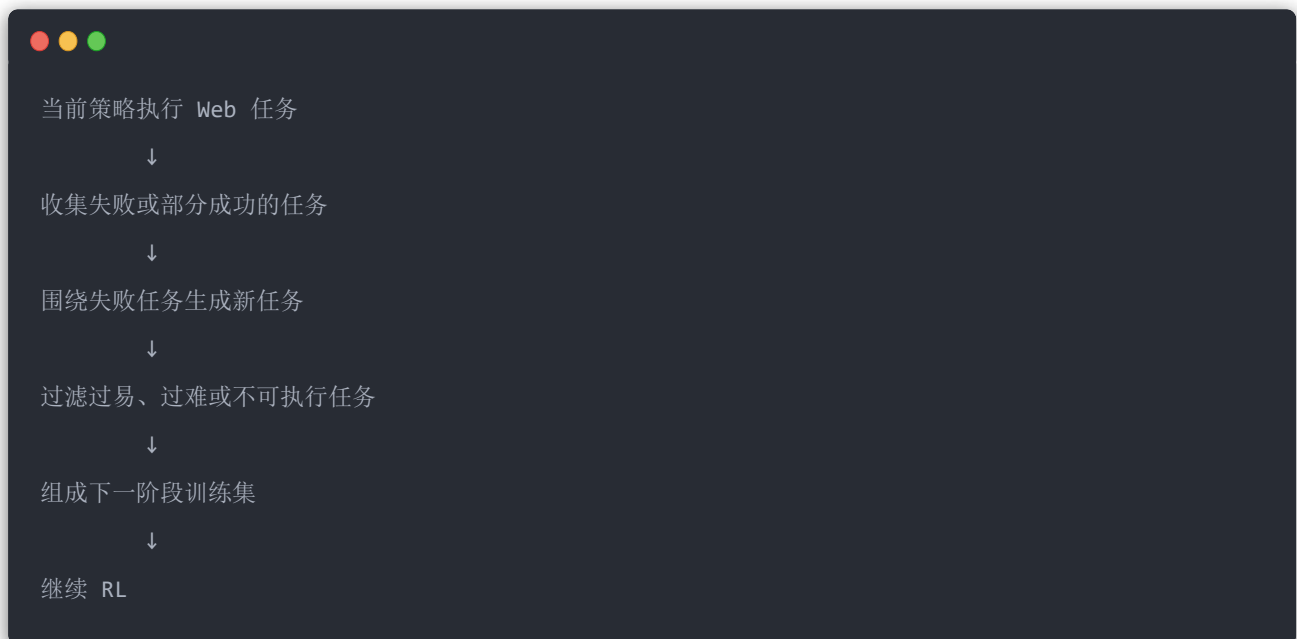
$$0.8^4 = 0.41, \quad 0.8^{16} = 0.028.$$

如果从 16 轮开始训练，成功 rollout 很少，组内可能全部是 0 reward，GRPO 无法得到有效优势。逐渐增加 horizon 可以显著缓解这个问题。这是一种真正作用于多轮轨迹结构的 curriculum。

2. WebRL：根据当前能力自动生成下一阶段任务

WebRL: Training LLM Web Agents via Self-Evolving Online Curriculum

Reinforcement Learning 是多轮 Web Agent 方向最典型的自演化课程学习工作。其循环为：



它不是预先人工排好 easy → medium → hard，而是让课程随模型能力变化。候选任务会通过价值函数估计初始状态难度，只保留成功概率处于某个中间区间的任务，例如论文使用类似：

$$0.05 < V_{\phi}(s_0) < 0.75.$$

于是：

- $V \approx 1$ ：已经会了，学习信号有限；
- $V \approx 0$ ：太难，几乎采不到成功轨迹；
- 中间区域：既能探索成功，也存在改进空间。WebRL 因此属于：

online multi-turn RL + competence-based curriculum.

它在 WebArena-Lite 上将 Llama-3.1-8B 成功率从 4.8% 提升至 42.4%，GLM-4-9B 从 6.1% 提升至约 43%。

3. R1-Code-Interpreter: 按“使用工具后的提升潜力”排课程

R1-Code-Interpreter 是多轮代码解释器调用中的 curriculum RL。它没有简单按照题目难度排序，而是定义 improvement potential：

$$\Delta_i = P(\text{correct} \mid \text{使用 Code Interpreter}) - P(\text{correct} \mid \text{不使用工具}).$$

训练流程大致是：

1. 先训练高 Δ_i 任务；
2. 模型先学会工具调用的基本闭环；
3. 再加入中等 Δ_i 的异构任务；
4. 最后处理工具收益较弱或信用分配更困难的任务。这比“按准确率排难度”更适合工具 Agent。因为一道题本身很难，不代表它适合当前工具 RL；真正重要的是模型能否通过多轮代码执行获得有效反馈。论文中，多阶段课程使 Qwen2.5 系列的平均 RL 增益从约 +3.4% 提升到 +9.3%。需要注意：它是多轮 Code Interpreter 训练中的多阶段任务课程，但不主要通过逐步增加 turn horizon 实现。

4. DRIVE: 困难度和 rollout budget 的联合课程

DRIVE 主要针对竞赛代码 RL，不是严格意义上的环境交互型多轮 Agent，但它的方法很容易扩展到多轮代码修复。其课程包含：

- 前期：大规模、难度较均匀的问题，每题约 8 个 rollout；
- 后期：少量高质量难题，每题约 64 个 rollout；
- 持续保留模型当前最难、但仍可探索成功的题目。可以理解为同时调节：

(task difficulty, rollout budget, context length).

对于多轮 RL，可以扩展成：

(task difficulty, max turns, rollouts per task).

例如，简单题只需 4 个 rollout、3 次修改；困难 CUDA Kernel 给 32 个 rollout、8 次性能优化机会。

5. ReVeal: 训练短轨迹，推理扩展到长轨迹

ReVeal 不是显式的课程学习论文，但非常相关。它训练代码生成—验证循环：

generate → self-verify → execute → revise.

训练只使用约 3 轮轨迹，但推理时可扩展到 20 轮以上。它说明如果局部的生成—验证—修正能力训练得足够好，多轮能力可以部分外推。不过它与 ScalingInter-RL 有一个关键区别：

- ReVeal：固定短 horizon 训练，测试时延长；
- ScalingInter-RL：训练过程中逐渐扩大 horizon。更理想的方案其实可以结合两者：

先学单轮验证

→ 再学 2~3 轮修复

→ 再学 4~8 轮长程自演化

→ 测试时扩展到 20+ 轮

6. Kevin：多轮 Kernel RL，但课程学习并非核心贡献

[Kevin](#) 明确训练多轮 CUDA 生成与优化：

$$\text{CUDA}_t \rightarrow \text{compile/profile feedback} \rightarrow \text{CUDA}_{t+1}.$$

论文发现，增加串行 refinement turns 通常比增加并行采样数量更有效。但严格来说，Kevin 的主要创新是：

- 多轮 trajectory RL；
- 长轨迹训练；
- 跨轮信用分配；
- 根据真实 CUDA 反馈连续优化。它并没有把“根据能力动态增加最大轮数”作为核心课程算法。因此可以说 Kevin 为多轮 CUDA RL 提供了环境和训练范式，但 ScalingInter-RL、WebRL 式的课程可以进一步加入 Kevin。

7. RAGEN：说明课程应该调哪些环境变量

[RAGEN](#) 提出 StarPO，并系统研究多轮 Agent RL 的训练稳定性。它发现多轮训练容易进入 Echo Trap：

模型产生某种无效行为

- 环境返回相似反馈
- 模型重复相似思路
- reward variance 突降
- 梯度出现尖峰
- 策略坍缩

其实验指出，多轮 rollout 更适合：

- 多样化的初始状态；
- 中等粒度的交互动作；
- 更频繁地重新采样；
- 过滤异常或退化轨迹；
- 使用更细粒度、reasoning-aware 的奖励。它本身不是完整课程算法，但这些结论可以转化为环境课程：

固定初态 → 多样初态 → 更随机环境，

或者：

粗粒度工具 → 中粒度操作 → 复杂组合操作。

多轮 RL 中主要有五种课程

课程维度	变化方式	代表工作
轨迹长度课程	短 horizon → 长 horizon	AgentGym-RL / ScalingInter-RL
能力匹配课程	选择当前成功率适中的任务	WebRL
提升潜力课程	优先训练工具最能带来提升的任务	R1-Code-Interpreter
任务难度课程	简单/均匀数据 → 困难数据	DRIVE
环境复杂度课程	少工具、确定环境 → 多工具、随机环境	RAGEN 等

其中最“纯粹”的多轮 RL 课程学习论文是：

1. **AgentGym-RL / ScalingInter-RL**：课程变量是交互 horizon；
2. **WebRL**：课程变量是随当前策略自演化的任务难度；

3. **R1-Code-Interpreter**: 课程变量是多轮工具使用的提升潜力。

如果用于 **CUDA Kernel 多轮优化**

我认为最合适的不是仅按 KernelBench Level 1/2/3 排序，而是设计一个二维或三维课程：

$C_i = (\text{initial correctness, optimization gap, required turns}).$

例如：

阶段	任务	最大轮数	奖励重点
1	修复编译错误	1-2	compile success
2	修复正确性错误	2-3	correctness
3	超过 PyTorch baseline	3-5	speedup > 1
4	需要 shared memory/fusion 的任务	5-8	speedup + rubric
5	多 Kernel 端到端优化	8-16	end-to-end latency

训练时动态选择成功率处于某个区间的任务：

$p_{\min} < \hat{P}_{\theta}(\text{success} \mid x, H) < p_{\max},$

比如只训练当前成功率在 10%–70% 的任务。全失败的任务先降级为更短子任务；全成功任务则增加性能门槛或交互复杂度。所以答案不只是“有”：多轮 RL 中的课程学习现在已经从传统的静态难度排序，发展到了动态 horizon、自演化任务、能力边界采样和 rollout-budget 联合调度。对于多轮 CUDA RL，现有研究仍没有把这些维度完全统一起来，这反而是一个相当明确的研究空间。